

Exercices de base

1. TP1 : Allumer une LED

- **Matériel** : Arduino Uno, 1 LED, 1 résistance (220 Ω), fils de connexion.
- **Principe** : brancher la LED sur une sortie numérique et l'allumer via le code.
- **Code minimal** :

```
int led = 13;
void setup() {
  pinMode(led, OUTPUT);
}
void loop() {
  digitalWrite(led, HIGH); // LED allumée
}
```

2. TP2 : LED clignotante

- **Matériel** : même que TP1.
- **Principe** : faire clignoter la LED avec un délai.
- **Code minimal** :

```
int led = 13;
void setup() {
  pinMode(led, OUTPUT);
}
void loop() {
  digitalWrite(led, HIGH); // ON
  delay(1000);             // attendre 1 seconde
  digitalWrite(led, LOW);  // OFF
  delay(1000);             // attendre 1 seconde
}
```

3. TP3 : Chenillard (séquence de LED)

- **Matériel** : Arduino Uno, 4 à 6 LED, résistances, fils.
- **Principe** : allumer les LED l'une après l'autre en boucle (effet "chenillard").
- **Code minimal** :

```
int leds[] = {2, 3, 4, 5}; // pins des LED
void setup() {
  for(int i=0; i<4; i++) {
    pinMode(leds[i], OUTPUT);
  }
}
void loop() {
  for(int i=0; i<4; i++) {
    digitalWrite(leds[i], HIGH);
    delay(300);
    digitalWrite(leds[i], LOW);
  }
}
```

Exercices intermédiaires

1. TP 1 : Contrôler un servomoteur avec Arduino

Matériel nécessaire

- 1 Arduino Uno
- 1 servomoteur (type SG90 ou similaire)
- Fils de connexion
- (Optionnel) 1 potentiomètre pour varier l'angle

Câblage

- **Fil rouge du servo** → 5 V Arduino
- **Fil marron/noir du servo** → GND Arduino
- **Fil orange/jaune du servo** → Pin numérique (ex. D9)

```
#include <Servo.h>

Servo monServo;

void setup() {
  monServo.attach(9); // servo connecté à la pin D9
}

void loop() {
  monServo.write(0);    // angle 0°
  delay(1000);
  monServo.write(90);   // angle 90°
  delay(1000);
  monServo.write(180);  // angle 180°
  delay(1000);
}
```

Variante avec potentiomètre

Si on ajoute un potentiomètre sur **A0**, on peut contrôler l'angle du servo manuellement :

```
#include <Servo.h>

Servo monServo;
int potPin = A0;

void setup() {
  monServo.attach(9);
}

void loop() {
  int val = analogRead(potPin); // lecture du potentiomètre (0-1023)
  int angle = map(val, 0, 1023, 0, 180); // conversion en angle
  monServo.write(angle); // servo prend l'angle
  delay(15);
}
```

2. TP2 : Capteur ultrason HC-SR04 et Arduino pour calculer la distance

Matériel nécessaire

- 1 Arduino Uno
- 1 capteur ultrason HC-SR04
- Fils de connexion
- (Optionnel) 1 LED ou buzzer pour signaler la distance

Câblage

- **VCC** → 5 V Arduino
- **GND** → GND Arduino
- **Trig** → Pin numérique (ex. D9)
- **Echo** → Pin numérique (ex. D10)

```

const int trigPin = 9;
const int echoPin = 10;
long duree;
int distance;

void setup() {
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
  Serial.begin(9600); // affichage sur le moniteur série
}

void loop() {
  // Envoi d'une impulsion
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  // Lecture du temps de retour
  duree = pulseIn(echoPin, HIGH);

  // Calcul de la distance (vitesse du son ~343 m/s)
  distance = duree * 0.034 / 2;

  // Affichage
  Serial.print("Distance: ");
  Serial.print(distance);
  Serial.println(" cm");

  delay(500);
}

```

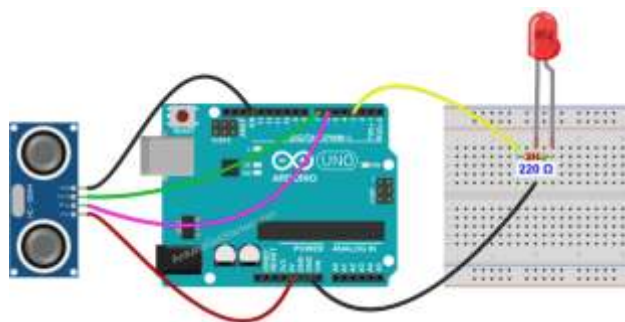
3. Capteur ultrason HC-SR04 et une LED qui s'allume si un obstacle est trop proche (par exemple à moins de 10 cm).

Matériel nécessaire

- Arduino Uno
- Capteur ultrason HC-SR04
- 1 LED + résistance (220 Ω)
- Fils de connexion

⚡ Câblage

- **HC-SR04 :**
 - VCC → 5 V
 - GND → GND
 - Trig → D9
 - Echo → D10
- **LED :**
 - Anode (+) → D3 via résistance
 - Cathode (-) → GND



```

const int trigPin = 9;
const int echoPin = 10;
const int ledPin = 3;

long duree;
int distance;

void setup() {
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
  pinMode(ledPin, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  // Envoi impulsion ultrason
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  // Lecture du retour
  duree = pulseIn(echoPin, HIGH);

  // Calcul distance en cm
  distance = duree * 0.034 / 2;

  // Affichage
  Serial.print("Distance: ");
  Serial.print(distance);
  Serial.println(" cm");

  // Condition : obstacle proche
  if (distance < 10) {
    digitalWrite(ledPin, HIGH); // LED ON
  } else {
    digitalWrite(ledPin, LOW); // LED OFF
  }

  delay(200);
}

```

Exercices avancés

1. TP1 : Arduino et capteur de température et d'humidité (DHT11 ou DHT22).

Matériel nécessaire

- Arduino Uno
- Capteur DHT11 ou DHT22
- Résistance (10 kΩ si nécessaire)
- Breadboard + fils de connexion
- (Optionnel) écran LCD 16x2 ou OLED pour affichage

✂ Câblage (DHT11/DHT22)

- VCC → 5 V Arduino
- GND → GND Arduino
- DATA → Pin numérique (ex. D2)
- Résistance de 10 kΩ entre DATA et VCC

```

#include "DHT.h"

#define DHTPIN 2      // Pin DATA du capteur
#define DHTTYPE DHT11 // ou DHT22

DHT dht(DHTPIN, DHTTYPE);

void setup() {
  Serial.begin(9600);
  dht.begin();
}

void loop() {
  float h = dht.readHumidity();
  float t = dht.readTemperature(); // en °C

  if (isnan(h) || isnan(t)) {
    Serial.println("Erreur de lecture du capteur !");
    return;
  }

  Serial.print("Humidité: ");
  Serial.print(h);
  Serial.print(" %\t");
  Serial.print("Température: ");
  Serial.print(t);
  Serial.println(" °C");

  delay(2000); // lecture toutes les 2 secondes
}

```

2. **TP : Extension Wi-Fi avec capteur de température et humidité (DHT11/DHT22), module comme ESP8266 ou ESP32.** Cela permet d'envoyer les données mesurées vers une application web, une base de données ou même un tableau de bord en ligne.

Matériel nécessaire

- Arduino Uno
- Capteur DHT11/DHT22
- Module Wi-Fi ESP8266 (ex. ESP-01) ou directement une carte ESP32 (qui intègre Wi-Fi et Bluetooth)
- Breadboard + fils de connexion

Principe

1. **Lecture des données** : Arduino lit température et humidité via le capteur DHT.
2. **Transmission Wi-Fi** : les données sont envoyées via le module ESP8266/ESP32.
3. **Affichage en ligne** : tu peux les consulter sur un serveur local, une application web (ex. ThingSpeak, Blynk, Firebase).

Exemple de code (ESP8266 + DHT11 vers ThingSpeak)

```

#include "DHT.h"
#include <ESP8266WiFi.h>

#define DHTPIN 2
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);

const char* ssid = "TonSSID";      // Nom du Wi-Fi
const char* password = "TonMDP";    // Mot de passe Wi-Fi
const char* server = "api.thingspeak.com";
String apiKey = "TA_CLE_API";

WiFiClient client;

void setup() {
  Serial.begin(115200);
  dht.begin();
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("Connecté au Wi-Fi !");
}

void loop() {
  float h = dht.readHumidity();
  float t = dht.readTemperature();

  if (isnan(h) || isnan(t)) {
    Serial.println("Erreur capteur !");
    return;
  }

  if (client.connect(server, 80)) {
    String postStr = "api_key=" + apiKey + "&field1=" + String(t) + "&field2=" + Str
    client.print("POST /update HTTP/1.1\n");
    client.print("Host: api.thingspeak.com\n");
    client.print("Connection: close\n");
    client.print("Content-Type: application/x-www-form-urlencoded\n");
    client.print("Content-Length: ");
    client.print(postStr.length());
    client.print("\n\n");
    client.print(postStr);
  }
  client.stop();

  delay(20000); // envoi toutes les 20 secondes
}

```

Ce TP avancé permet :

- De comprendre la **connexion Wi-Fi** avec Arduino/ESP.
- De **publier des données capteur en ligne**.
- De créer une **station météo connectée** accessible depuis n'importe où.